Procedural World Generation, Spatial Epidemic Simulation and Real-Time Rendering for a Browser Strategy Game

Algorithms, Mathematical Models and Performance Engineering

Kazak Vladimir

Technical Report

November 2026

Abstract

This report describes the technical design of a real-time strategy game in which a spreading organism colonises a procedurally generated planet while the planet's societies detect it and race to stop it. We present (i) a deterministic world-generation pipeline that turns a single integer seed into continents, regions, climates and a multimodal transport network, combining value noise, fractional Brownian motion, order statistics, connected-component labelling, farthest-point sampling, a noisy multi-source Dijkstra partition, union–find connectivity repair and weighted A* routing; (ii) a discrete-time stochastic metapopulation model with logistic growth inside regions, link-based transmission, an endogenous detection and countermeasure process with adaptive difficulty, and containment behaviour; (iii) a sub-regional visual model that renders the colony from one scalar per region through a per-cell threshold field evaluated on the GPU; and (iv) the performance engineering that keeps generation, simulation and rendering within a browser's frame budget: time-sliced generators, linear-time selection, tiled texture uploads, GPU offloading and delivery-size optimisation. The methods are illustrated with the output of the implementation and validated by determinism checks, headless simulated players and runtime measurements.

Keywords: procedural content generation; value noise; fractional Brownian motion; farthest-point sampling; Dijkstra partition; A* search; logistic growth; metapopulation model; GPU rendering; real-time systems.

Contents

1 Introduction **3**
1.1 Problem and objectives . . . 3
1.2 Structure of the report . . . 3

2 System architecture **3**

3 Procedural world generation **4**
3.1 Deterministic pseudo-random source . . . 4
3.2 Value noise and fractional Brownian motion . . . 4
3.3 Domain shaping . . . 5
3.4 Sea level as an order statistic . . . 6
3.5 Landmass extraction . . . 7
3.6 Region seeding by farthest-point sampling . . . 7
3.7 Region partition by noisy multi-source Dijkstra . . . 8
3.8 Region attributes . . . 8
3.9 Transport network . . . 9
3.10 Maritime routing . . . 10
3.11 Determinism and incremental evaluation . . . 11

4 Epidemic simulation **11**
4.1 Time and pacing . . . 11
4.2 Growth inside a region . . . 12
4.3 Transmission between regions . . . 13
4.4 Sub-regional visual model . . . 13
4.5 Detection and countermeasure . . . 13
4.6 Containment: closures and laboratories . . . 15
4.7 Endgame . . . 15
4.8 Player economy . . . 15
4.9 Outcomes . . . 16

5 Rendering **16**
5.1 Base map . . . 16
5.2 GPU colony shader . . . 16
5.3 3D terrain and procedural snow . . . 16

6 Performance engineering **17**
6.1 Methodology . . . 17
6.2 Time-sliced generation . . . 17
6.3 Linear-time selection . . . 17
6.4 Tiled texture uploads . . . 17
6.5 Further measures . . . 18
6.6 Delivery size . . . 18

7 Validation **19**
7.1 Determinism . . . 19
7.2 Balance with simulated players . . . 19
7.3 Runtime . . . 19

8 Discussion and limitations **19**

9 Conclusion **19**

A Symbols **21**

B Supplementary animations **22**

1 Introduction

1.1 Problem and objectives

Games about the spread of an infection over a world map, while an opposing force develops countermeasures, combine three technical problems: the world on which the spread happens, the dynamics of the spread and the response, and a presentation that makes both legible in real time. The system described here targets short sessions in a web browser and pursues three objectives:

1. **Replayability without authored content.** Every session takes place on a new world generated from a seed; the generator must produce varied yet always playable worlds.
2. **Legible, deterministic rules.** The simulation must be simple enough to reason about, deterministic for a given seed and sequence of player decisions, and cheap enough to be run many thousands of times for automated balancing.
3. **Fluid presentation within tight budgets.** Generation, simulation and rendering must fit a real-time frame budget on modest hardware, and the delivered application must fit a small download budget.

1.2 Structure of the report

Section 2 outlines the architecture. Section 3 presents world generation, Section 4 the simulation, Section 5 the rendering, and Section 6 the performance engineering. Section 7 describes how the system is validated; Section 8 discusses limitations. A table of symbols is given in Appendix A, and Appendix B contains animations of the generation and the simulation.

2 System architecture

The system is implemented in C# on the Unity engine and delivered to browsers through WebGL. Its code is split into three layers with one-way dependencies (Fig. 1). The *world generator* is a pure function of the seed. The *rules* hold the state of one session and advance it by whole simulated days through a single entry point; they use plain numeric data only and no engine objects. The *presentation* (map views, interface screens and GPU shaders) reads that state and draws it. Because the two lower layers are independent of the engine, the same rules run inside the application and in a command-line harness used for automated balancing and measurement.

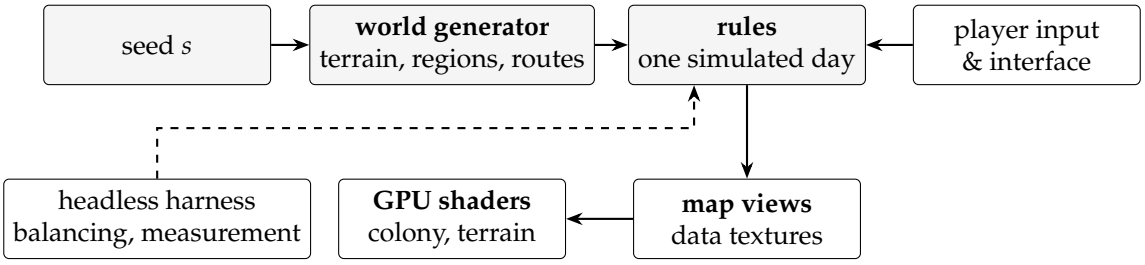


Figure 1: Layered architecture. Solid arrows: data flow during play; dashed: the same rules driven by a command-line harness without the engine.

The world is a regular grid of $W \times H$ cells; cell c has integer coordinates (x_c, y_c) and index $c = y_c W + x_c$. Two grid resolutions are supported, a standard one and a larger one for an extended mode, with $N = WH$ cells in total.

3 Procedural world generation

The generator executes the pipeline of Algorithm 1. All random decisions draw from one deterministic stream, so the output is a function of the seed alone.

Algorithm 1: World generation pipeline

Input: seed s, grid resolution
Output: world: heights, land mask, regions, attributes, transport network

1 initialise the pseudo-random stream from s; draw a geography and a society preset;
2 compute the height field from fractional Brownian motion with edge attenuation (Sections 3.2 and 3.3);
3 $\theta \leftarrow$ the order statistic leaving the target land share above it; land $\leftarrow h > \theta$ (Section 3.4);
4 label connected landmasses and keep the significant ones (Section 3.5);
5 place region centres by farthest-point sampling (Section 3.6);
6 partition land by multi-source Dijkstra with noisy costs (Section 3.7);
7 assign region attributes (Section 3.8);
8 build land, sea and air links and repair connectivity with union–find (Section 3.9);
9 route sea lanes with weighted A* and smooth them (Section 3.10);

3.1 Deterministic pseudo-random source

Randomness is supplied by *mulberry32*, a 32-bit counter-based generator. With state a and arithmetic modulo 2^{32},

$$\begin{aligned} a &\leftarrow a + \texttt{0x6D2B79F5}, \\ t &\leftarrow \big(a \oplus (a \gg 15)\big) \cdot (a \mid 1), \\ t &\leftarrow \Big(t + \big(t \oplus (t \gg 7)\big) \cdot (t \mid 61)\Big) \oplus t, \\ u &= \big(t \oplus (t \gg 14)\big)/2^{32} \in [0,1). \end{aligned} \tag{1}$$

Because the increment is odd, the state visits all 2^{32} values before repeating. The generator is fast and fully portable, so the same seed yields the same world on every platform. Figure 2 shows a sample of its output: the histogram is flat (a χ^2 test over equal bins is consistent with uniformity) and successive pairs fill the unit square without visible structure.

3.2 Value noise and fractional Brownian motion

Terrain is built from *value noise* [1, 2]. A table P of random values is filled from the stream, and integer lattice points are hashed into it with two large odd multipliers μ_1, μ_2:

$$L(i,j) = P\big[(\mu_1 i \oplus \mu_2 j) \bmod |P|\big]. \tag{2}$$

Between lattice points, the four surrounding values $a = L(i,j)$, $b = L(i{+}1,j)$, $c = L(i,j{+}1)$, $d = L(i{+}1,j{+}1)$ are blended with the smoothstep weights $s(t) = t^2(3-2t)$. Their derivative vanishes at $t \in \{0,1\}$, so the surface has no creases along lattice lines:

$$N(x,y) = a + (b-a)\,s_x + (c-a)\,s_y + (a-b-c+d)\,s_x s_y, \qquad s_x = s(x - \lfloor x \rfloor),\ s_y = s(y - \lfloor y \rfloor). \tag{3}$$

A single layer produces smooth, blob-like relief (Fig. 3). Natural coastlines are statistically self-similar [3], so several octaves are summed with doubling frequency and halving amplitude,

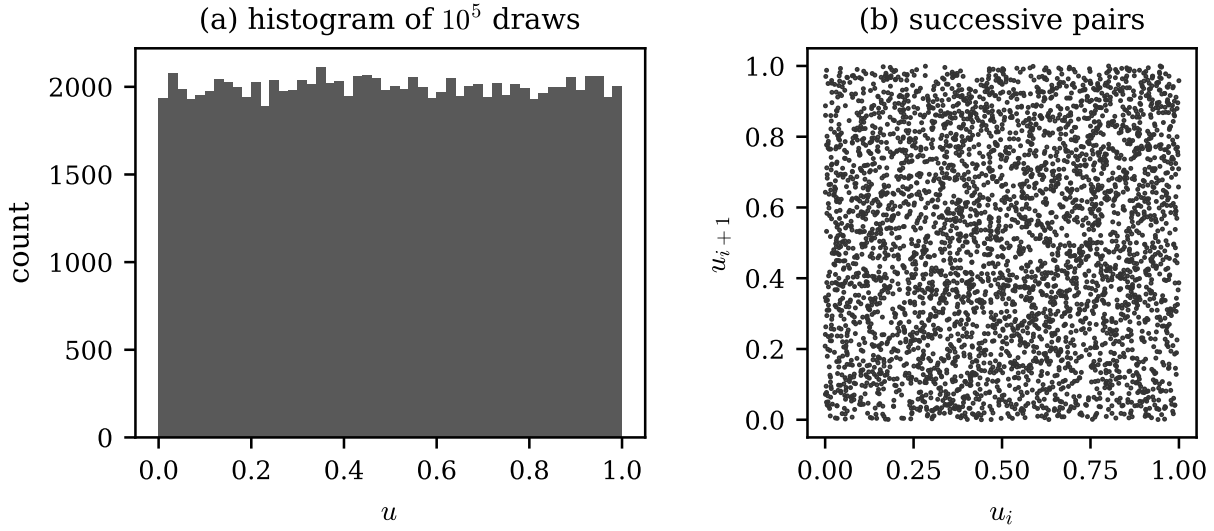


Figure 2: Output of the generator. (a) Distribution of 10^5 values. (b) Successive pairs (u_i, u_{i+1}).

a fractional Brownian motion (fBm) [4]:

$$F(x,y) = \frac{\sum_{o=0}^{O-1} 2^{-(o+1)}\, N(2^o x, 2^o y)}{\sum_{o=0}^{O-1} 2^{-(o+1)}}. \tag{4}$$

The normalisation keeps $F \in [0,1]$. Low octaves set the continents, high octaves add bays and capes (Fig. 4).

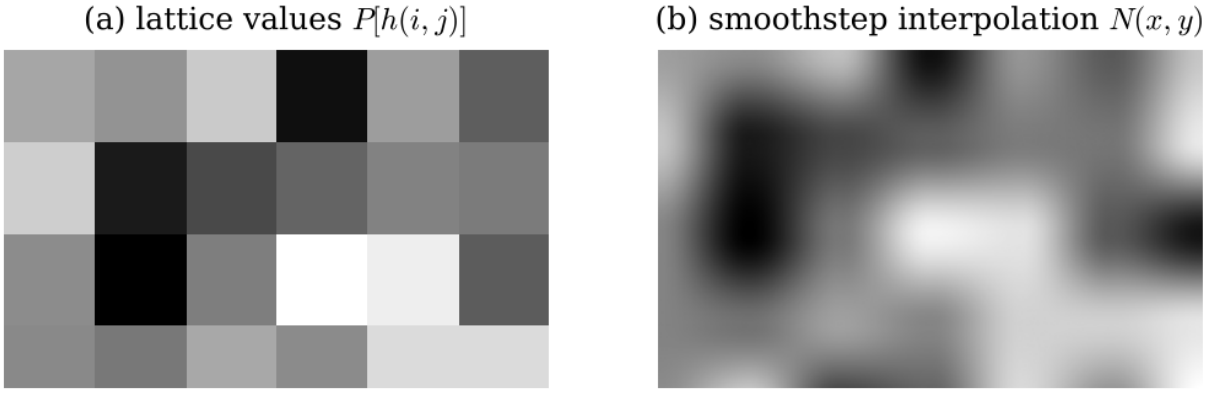


Figure 3: Value noise. (a) Random lattice values. (b) The same values blended with Eq. (3).

3.3 Domain shaping

The height of a cell samples the fBm in a domain scaled by a frequency ϕ, stretched horizontally by an aspect factor α and offset by a random vector (o_x, o_y), then corrected near the map border and, optionally, raised towards the centre:

$$h(x,y) = F\Big(\alpha\phi\,\tfrac{x}{W} + o_x,\ \phi\,\tfrac{y}{H} + o_y\Big) - \kappa\,(1-e)^2 + \gamma\big(1 - \lambda_c\, r\big), \tag{5}$$

where r is the normalised distance from the centre and $e \in [0,1]$ the edge proximity

$$e = \min\Big(1,\ \min\big(\tfrac{\min(x, W-1-x)}{m_x W},\ \tfrac{\min(y, H-1-y)}{m_y H}\big)\Big) \tag{6}$$

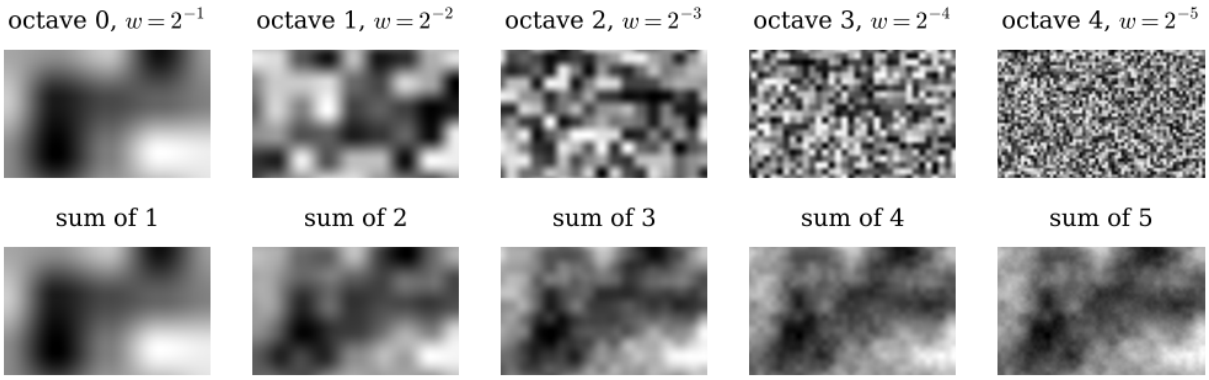


Figure 4: Fractional Brownian motion. Top: individual octaves with weights $2^{-(o+1)}$. Bottom: partial sums of Eq. (4).

with relative margins m_x, m_y. The quadratic attenuation leaves the interior untouched and depresses only the margins, steeply at the border, so every world is surrounded by navigable sea (Fig. 5). The centre term γ is used by presets that favour a single supercontinent. Larger grids scale ϕ with the grid size, which keeps the size of individual features constant while their number grows.

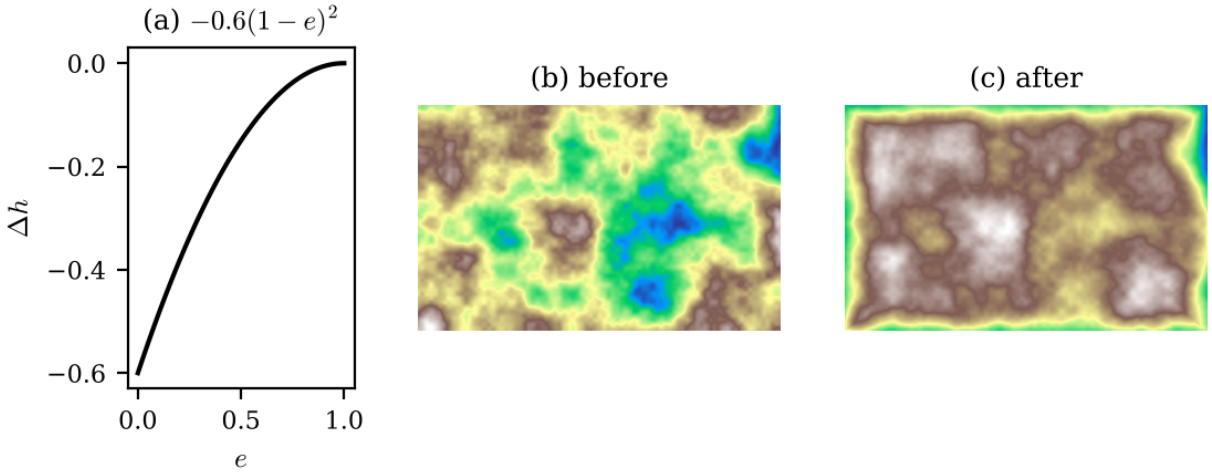


Figure 5: Edge attenuation. (a) Profile of the correction. (b,c) A height field before and after.

3.4 Sea level as an order statistic

Instead of fixing a water level, the generator fixes the land share λ requested by the geography preset and solves for the level. The sea level is the order statistic

$$\theta = h_{(k)}, \qquad k = \lfloor N(1-\lambda) \rfloor, \tag{7}$$

so that exactly a fraction λ of the cells lies above it (Fig. 6). Sorting would cost $O(N \log N)$. The implementation uses Hoare's selection algorithm [5] (quickselect), which partitions around a pivot and recurses only into the side that contains position k, giving expected $O(N)$ time and the identical result. Border cells are forced to water.

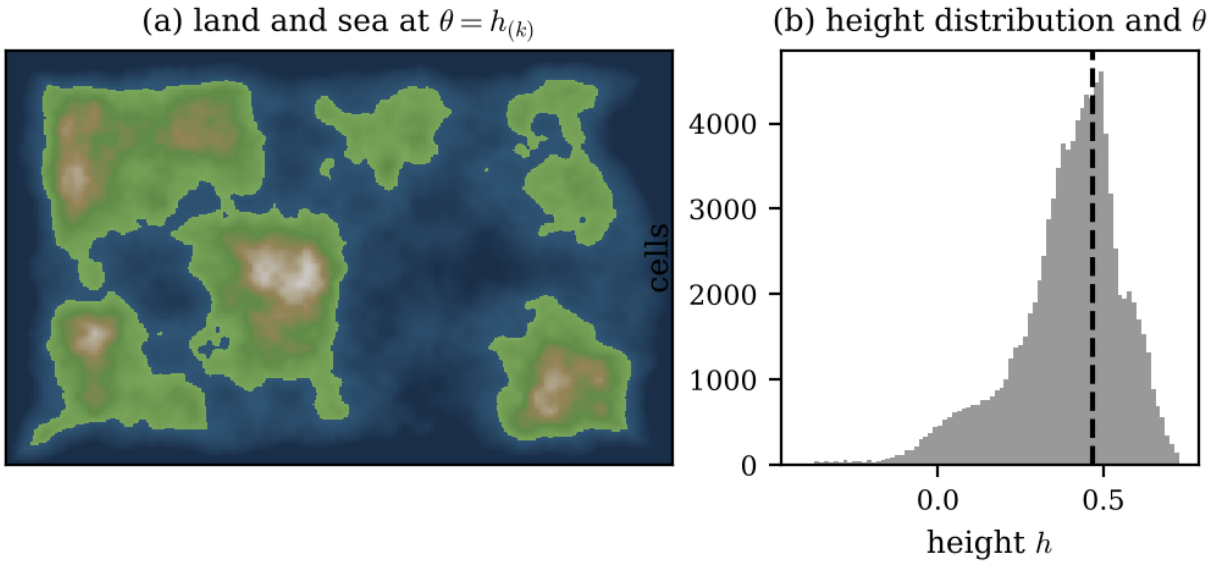


Figure 6: Sea level. (a) Resulting land and sea. (b) Height distribution with the threshold θ.

3.5 Landmass extraction

Connected components of the land mask under 4-connectivity are labelled by breadth-first search [6]: each unvisited land cell seeds a queue that floods its component, in $O(N)$ time overall. Components are sorted by size, and only the largest ones above a minimum size are kept (the largest is always kept), so that no region is created on an insignificant islet (Fig. 7).



Figure 7: Landmasses labelled by breadth-first search. Grey: components discarded by the size filter.

3.6 Region seeding by farthest-point sampling

The land is divided into N_R regions, with N_R drawn from a range that grows with the grid resolution. Landmass C receives $n_C = \max\big(1, \mathrm{round}\big(N_R|C|/|L|\big)\big)$ centres, $|L|$ being the total land area. Within each landmass the centres are placed by *farthest-point sampling* [7, 8]: starting from a random cell, each new centre is the cell that maximises the distance to all previous centres,

$$s_{k+1} = \arg\max_{p \in C} \min_{j \le k} \|p - s_j\|^2. \tag{8}$$

The inner minimum is maintained incrementally (Algorithm 2), so each step is a single pass over C and the total cost is $O(n_C|C|)$. The greedy rule is within a factor of two of the optimal covering radius [7]. It yields well-spread but irregular centres (Fig. 8).

Algorithm 2: Farthest-point sampling inside one landmass

$D[p] \leftarrow \infty$ for all $p \in C$; $s \leftarrow$ random cell of C;
for $k \leftarrow 1$ **to** n_C **do**
  emit centre s;
  foreach $p \in C$ **do** $D[p] \leftarrow \min\big(D[p], \|p - s\|^2\big)$;
  $s \leftarrow \arg\max_{p \in C} D[p]$;

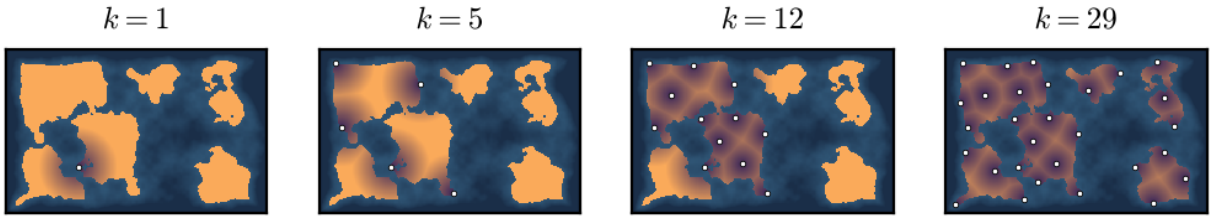


Figure 8: Farthest-point sampling after k centres. Brightness: distance to the nearest centre; the next centre is placed at the brightest cell of its landmass.

3.7 Region partition by noisy multi-source Dijkstra

Every land cell is assigned to the centre with the smallest *geodesic* cost, computed for all centres at once by a multi-source Dijkstra search [9] with a binary heap. Entering cell q costs

$$w(q) = 1 + \beta_w \, N(\nu x_q + o_x, \; \nu y_q + o_y), \tag{9}$$

with a low noise frequency ν and weight β_w. The region of q is $\arg\min_k d_k(q)$, where d_k is the cheapest path cost from centre k over land. With constant w the partition reduces to a discrete Voronoi diagram [10] with straight bisector borders. The noise term makes some ground more expensive than other ground, so borders follow valleys of the noise and acquire the meanders of natural boundaries (Fig. 10). Each cell is settled once, giving $O(N \log N)$. Figure 9 shows the growing fronts (Animation S1).

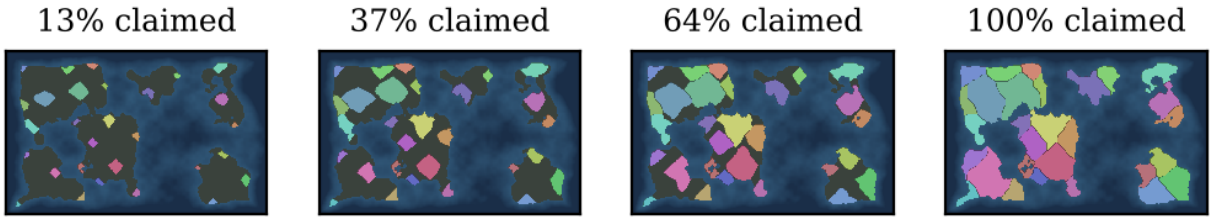


Figure 9: Multi-source Dijkstra growth at four stages (Animation S1 in Appendix B).

3.8 Region attributes

For each region the generator computes the area A, the centroid $(\bar{x}, \bar{y})$ and a label cell (the cell closest to the centroid), and draws its attributes:

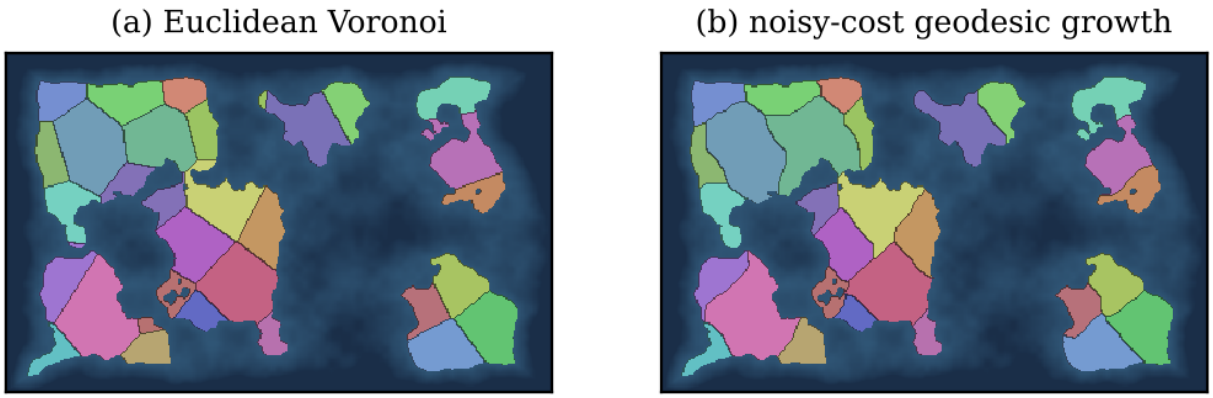


Figure 10: Same centres, two partitions. (a) Nearest centre in the Euclidean metric. (b) Noisy geodesic metric, Eq. (9).

- **Climate** from latitude, $\ell = 2\,|\bar{y}/H - \frac{1}{2}|$ plus a small random jitter: cold above a threshold τ_c, hot below τ_h, temperate in between (Fig. 11).
- **Wealth** (rich, middle, poor) and **density** (dense or sparse), drawn from categorical distributions.
- **Population** proportional to area, scaled by density, by a random factor, and reduced in cold regions.
- **Paranoia** π, the level of global spread at which the region begins to close its borders (Section 4.6), drawn uniformly and lowered for rich and isolated regions.

A world-level *society* preset modulates several of these quantities globally, for example flight density, detection speed or research speed, so that worlds differ in character as well as in shape.

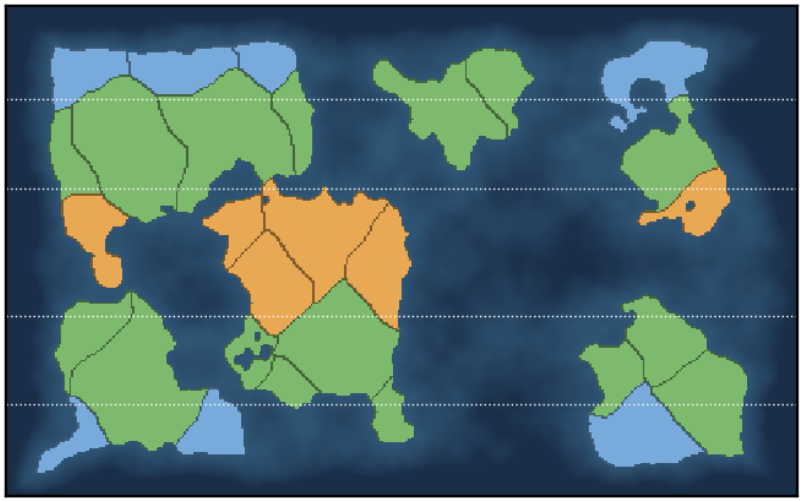


Figure 11: Climate zones (blue cold, green temperate, orange hot) with the latitude thresholds τ_c, τ_h (dotted).

3.9 Transport network

Three kinds of link form a multigraph over the regions (Fig. 12):

- **Land links** join every pair of regions that share a border.

- **Sea links.** Regions touching the *open* ocean receive a port with some probability. Open ocean is the water connected to the map edge, found by a flood fill from the border, so inland lakes are excluded. Every landmass with a coast is guaranteed at least one port, and each port is linked to its nearest ports that are not land neighbours.
- **Air links.** Airports appear with a probability that increases with wealth and density, and each receives a small random number of flights to other airports, drawn as quadratic Bézier arcs.

Finally the generator guarantees that the network is connected. A union–find structure with path compression [11] merges the endpoints of all links. While more than one component remains, the closest pair of coastal regions in different components is joined by a new sea link, creating ports where needed. No generated world can therefore be unwinnable by construction.

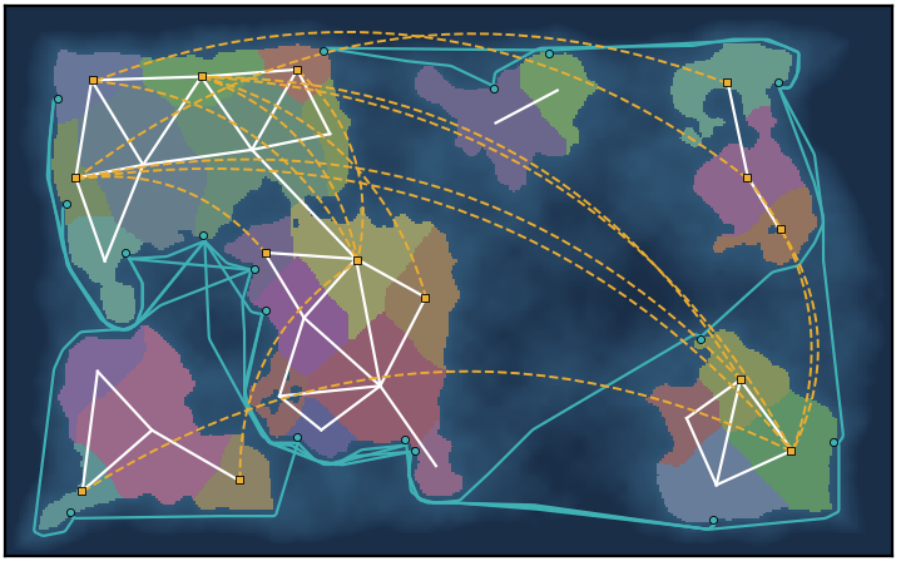


Figure 12: Transport network: land links (white), sea lanes (teal, ports as circles) and flights (orange dashed, airports as squares).

3.10 Maritime routing

Sea lanes must pass around landmasses and keep away from coasts. A truncated distance transform of the water is computed first, by a multi-source breadth-first search from all land cells over the 8-neighbourhood: $\varsigma(c)$ is the Chebyshev distance of water cell c to land, saturating at a small cap (Fig. 13a). Lanes are then found by A* [12] on the 8-connected water grid with step cost

$$c(p \to q) = \big(\sqrt{2} \text{ if diagonal, } 1 \text{ otherwise}\big) \cdot \omega(\varsigma_q), \qquad \omega \geq 1 \text{ decreasing with } \varsigma, \tag{10}$$

and the octile heuristic

$$\hat{h}(q) = \max(\Delta x, \Delta y) + (\sqrt{2} - 1)\min(\Delta x, \Delta y). \tag{11}$$

Since $\omega \geq 1$, $\hat{h}$ never exceeds the true remaining cost; it is admissible, so A* returns a minimum-cost lane [12]. The penalty ω pushes lanes offshore without forbidding narrow straits. The cell path is thinned and then smoothed by repeated passes of Chaikin's corner cutting [13], which replaces each segment P_iP_{i+1} by the points $\frac{3}{4}P_i + \frac{1}{4}P_{i+1}$ and $\frac{1}{4}P_i + \frac{3}{4}P_{i+1}$ and converges to the quadratic B-spline of the polygon (Fig. 14). Thanks to the heuristic, A* explores only a small fraction of the water cells (Fig. 13b).

(a) distance to shore ς (b) A* expansion order and lane



Figure 13: Sea-lane routing, cropped to the search area. (a) Distance-to-shore field ς (red: adjacent to land). (b) Cells expanded by A* in order (pink to yellow) and the smoothed lane (white).

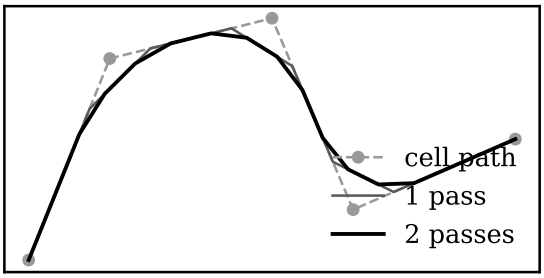


Figure 14: Chaikin corner cutting: one and two passes.

3.11 Determinism and incremental evaluation

The pipeline is written as a C# iterator that *yields* frequently inside its loops (per batch of rows, per batch of heap operations, per sea lane). Its caller resumes it until a fixed time budget is used up and then returns control to the engine (Section 6.2). Running the iterator to completion produces exactly the same world as a monolithic call; this was verified by hashing the region map and every sea lane for several seeds at both grid resolutions.

4 Epidemic simulation

The simulation is a discrete-time stochastic *metapopulation* model [14]: each region is a well-mixed population, and regions interact only through the transport links. One simulated day is advanced in the fixed order of Algorithm 3.

4.1 Time and pacing

All rates are multiplied by a global pace factor ρ, and every interval specified at unit pace is converted to simulated days by $\mathrm{Days}(d) = \mathrm{round}(d/\rho)$. This single factor sets the length of a session without changing the relative balance of any mechanic.

Algorithm 3: One simulated day

foreach *reached region i that is not saturated* **do**
  grow I_i logistically (Eq. (12)); mark it overrun near saturation
foreach *link (a, b) and both directions* **do**
  with probability p_{ab} (Eq. (15)) launch a carrier $a \to b$
advance carriers; an arriving carrier infects its target (Eq. (16));
if *research has not started* **then**
  raise suspicion A (Eq. (18)); start research when it saturates
else
  raise the countermeasure C (Eq. (19)); close links by paranoia; open laboratories
spawn collectible rewards and random events; in the endgame, seed remaining regions;
if *every region is overrun* **then** the player wins;
if C *is complete* **then** the player loses;

4.2 Growth inside a region

Let $I_i(t)$ be the infected population of region i and P_i its population. Growth follows the discrete logistic map [15]:

$$I_i(t+1) = I_i(t) + b_i(t)\, I_i(t)\Big(1 - \frac{I_i(t)}{P_i}\Big), \tag{12}$$

$$b_i(t) = \beta\, \rho\, g\, m_g\, r_{\text{clim}}\, r_{\text{wealth}}\, r_{\text{dense}}\, S_i(t)\, B_i(t), \tag{13}$$

where β is the base rate, g a difficulty factor, m_g the product of the player's growth upgrades, and the r factors express how hospitable each region type is (upgrades can raise them). The seasonal factor

$$S_i(t) = 1 + \varepsilon_i\, \sigma \sin(2\pi t / Y), \qquad \varepsilon_i \in \{+1,\, 0,\, -\varepsilon_h\} \text{ for cold, temperate, hot regions,} \tag{14}$$

favours cold regions in summer and hot regions in winter. $B_i(t)$ collects temporary multipliers (special actions, events, the endgame of Section 4.7). A region is *overrun* when I_i comes within a small margin of P_i; it then saturates, rewards the player and destroys any laboratory in it. The trajectory has the familiar sigmoid shape: slow start, near-exponential middle, saturation (Fig. 15).

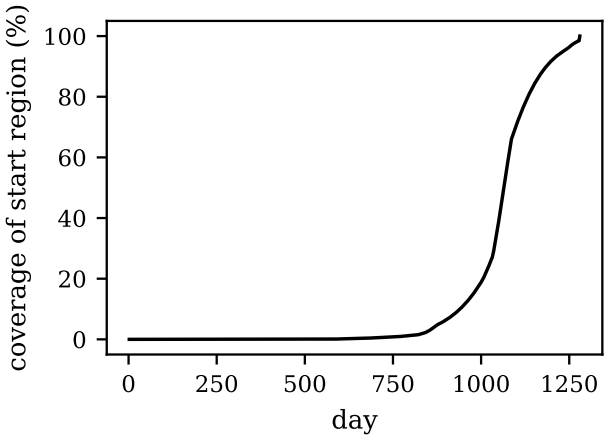


Figure 15: Coverage I/P of a starting region during a simulated session.

4.3 Transmission between regions

Every day each link is tried in both directions. A jump from a to b requires that the source has passed a minimum infected count, that the target is clean, and that no carrier is already heading to it. It then happens with probability

$$p_{ab} = \tau_{\text{type}}\,\rho\,\kappa\,f_a^{1/4}\,m_{\text{type}}\;\cdot\;\begin{cases}1 & \text{link open}\\ \ell_{\text{type}} & \text{link closed}\end{cases},\qquad f_a = I_a/P_a, \tag{15}$$

with a base rate τ per transport type, a difficulty factor κ, upgrade multipliers m, and a *leak* factor $\ell < 1$ for closed links. A link counts as open when the target keeps that transport open and the source either keeps it open or is already heavily infected. The sub-linear exponent makes even lightly infected regions leak noticeably, which keeps the early game moving. A success launches a *carrier* whose travel time is constant for land and proportional to route length (clamped to a range) for sea and air. On arrival it infects the target at an *entry cell*: the port, the airport, or for land the target cell nearest to the source. Figure 16 shows the spread over a session (Animation S2).

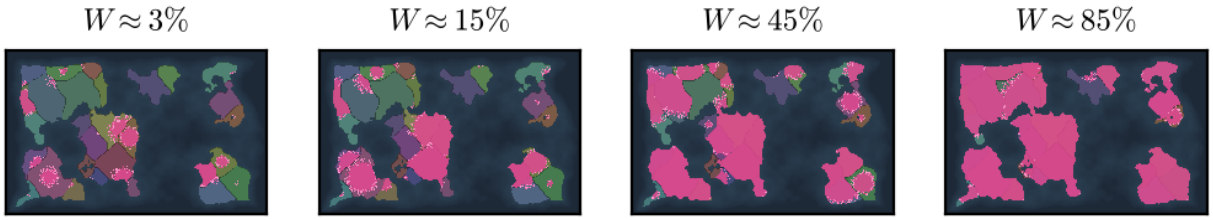


Figure 16: Spread over a simulated session at increasing world coverage W, drawn with the threshold model of Section 4.4 (Animation S2).

4.4 Sub-regional visual model

The rules track only one number per region, $f_i = I_i/P_i$. To show a colony that creeps outwards from where it arrived, each cell c of a newly reached region receives a fixed threshold at the moment of infection:

$$\vartheta_c = (1-\eta)\,\frac{\|c - e_i\|}{\max_{c' \in i}\|c' - e_i\|} + \eta\,\xi_c, \tag{16}$$

where e_i is the entry cell, $\xi_c \sim \mathcal{U}(0,1)$ a per-cell grain fixed at world creation and η the grain weight. The region's visibility is a concave function of coverage,

$$v_i = v_0 + (1-v_0)\,f_i^{1/3},\qquad \text{and cell } c \text{ is drawn as colonised } \iff v_i > \vartheta_c. \tag{17}$$

Because v_i is non-decreasing in f_i and ϑ_c is fixed, the colonised set only grows. It expands roughly as a disc around e_i with a ragged front whose roughness is set by η (Fig. 17). The cube root makes small infections visible early. The comparison is evaluated per pixel on the GPU (Section 5.2); the CPU only supplies v_i.

4.5 Detection and countermeasure

Before research starts, a suspicion level A grows each day with the world coverage $W = \sum_i I_i / \sum_i P_i$:

$$A \leftarrow A + \rho\,\alpha\,m_{\text{vis}}\,(1 + \chi_A\,\nu)\,\left(a_0 + a_1\sqrt{W}\right), \tag{18}$$

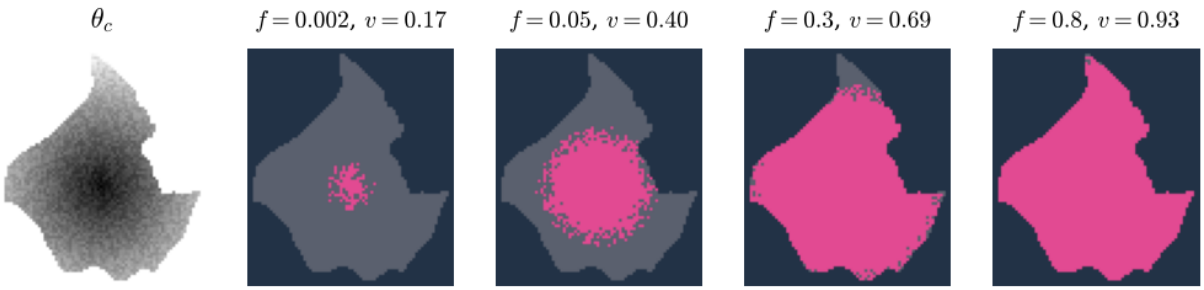


Figure 17: The threshold field ϑ_c of one region (dark: low) and the drawn colony for increasing coverage f (Animation S3).

where v is the colony's *noise*, a trade-off chosen by the player through upgrades (Section 4.8). Research begins when A saturates or when W passes a notice level. From then on the countermeasure C advances as

$$C \leftarrow C + \rho\, c_0\, c\,(1 + \chi_C\, v)\; E\; \big(h_0 + (1 - h_0)\,\mathcal{H}\big)\; m_C\,(1 + \lambda_{\text{lab}}\, n_{\text{lab}}), \tag{19}$$

where n_{lab} is the number of open laboratories, λ_{lab} their strength, and $\mathcal{H}$ the wealth-weighted share of the world that is still healthy:

$$\mathcal{H} = \frac{\sum_i w_i (1 - f_i)}{\sum_i w_i}, \qquad w_i \text{ increasing with wealth.} \tag{20}$$

The factor E is an adaptive *rubber band*. With the lead $\Delta = C - 100\,W$ (in percentage points) and n_b the number of upgrades bought,

$$E = \begin{cases} \max\big(1 - a\,\min(1, n_b / n^*),\ 1 - \delta(\Delta - \Delta_+)\big) & \Delta > \Delta_+ \\ 1 + \chi\,\min\big(1, (-\Delta - \Delta_-)/\Delta_s\big) & \Delta < -\Delta_- \\ 1 & \text{otherwise.} \end{cases} \tag{21}$$

Research slows down when it is far ahead and speeds up when the player is far ahead, which keeps sessions competitive across skill levels without erasing the reward for good play. Figure 18 shows W, A and C over a simulated session.

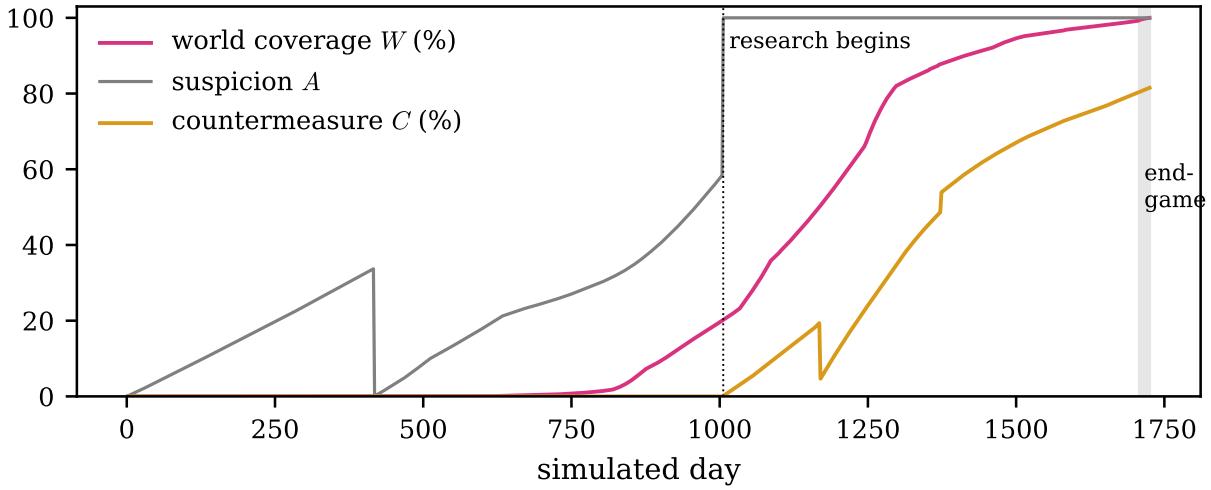


Figure 18: World coverage W, suspicion A and countermeasure C over a simulated session. The dotted line marks the start of research; the shaded band is the endgame.

4.6 Containment: closures and laboratories

Once research runs, every region compares world coverage with its paranoia π_i, scaled by difficulty and by the player's upgrades. It closes its airport, then its port, then its land borders at increasing multiples of π_i, one closure per region per day (Fig. 19). Above a collapse level of world coverage, containment breaks down: no further closures happen and closed links stop holding. Laboratories open at random intervals up to a maximum number. Their host is drawn by roulette-wheel selection with weights that favour rich, healthy regions, and each open laboratory accelerates the countermeasure. The player can wreck a laboratory in a reached region, and overrunning its region destroys it.



Figure 19: Containment during research: closed airports (orange ×), closed ports (teal ×), closed land borders (red rings) and laboratories (white triangles).

4.7 Endgame

When only a small number of regions (a fixed fraction of N_R, with a minimum) remain unsaturated, an *endgame* phase begins: growth is multiplied and, at regular intervals, a random unreached region is infected directly. This removes the long tail in which the last isolated regions would take a very long time to fall, while still requiring the player to have won the race against the countermeasure.

4.8 Player economy

Collectibles. The player's currency comes from collectibles that appear in reached regions at random intervals that shorten as the colony spreads. They must be collected within a short time; a missed one pays only a fraction. Reaching and overrunning regions also pay.

Upgrades. Upgrades are arranged in several hexagonal trees with prerequisites. The price of upgrade n is

$$\text{price}_n = \text{round}\big((c_n + q\,O)\,\kappa_{\text{cost}}\big) + s_n\,o_n, \tag{22}$$

where c_n is its base price, O the total number of upgrades owned (so that every purchase makes the next one dearer), q the per-purchase increment, and s_n an optional step applied per copy o_n already owned. Upgrades act multiplicatively on the factors of Eqs. (13), (15), (18) and (19). Selling one refunds part of its price. The noise ν is a deliberate trade-off: noisy upgrades make collectibles more frequent but also raise suspicion and research speed.

Mutations and variants. When the number of reached regions passes successive milestones, the player drafts one of three random mutations; redrawing costs an increasing price. Several organism variants, unlocked by progress, change passive factors and provide a special action with a cost, a cooldown and a duration.

4.9 Outcomes

The player wins when every region is overrun and loses when the countermeasure completes. Apart from upgrade effects both W and C are monotone, so a session is a race between the logistic spread and the research process, modulated by containment and the rubber band.

5 Rendering

5.1 Base map

At world creation the CPU paints a base texture at a fixed number of pixels per cell, reduced on low-end devices. It contains sea depth bands from a breadth-first distance to land, biomes chosen by climate and height, terraced relief, coastline ink, borders, rivers and decorative vegetation. Painting proceeds in row batches inside the same time-sliced iterator as generation.

5.2 GPU colony shader

The colony is not painted by the CPU. Three data textures encode everything the shader needs:

- a *cell* texture with the threshold ϑ_c (16-bit) and the region index of each cell, including a ring of coastal water so the colony can stain the shoreline;
- a *static* texture with the land mask, grain and relief;
- a *region* texture, a single row with the visibility v_i and state flags (overrun, closed, selected, hovered) of every region.

Each frame the CPU uploads only the region row. The fragment shader samples the cell texture, looks up the region and evaluates Eq. (17) with soft edges, outlines and a glow at the colony front. The cell texture changes only when a region is first reached (Section 6.4).

5.3 3D terrain and procedural snow

In 3D mode a coarse height mesh is displaced by the terrain heights, and a single terrain shader composes land, colony, snow, ink and outlines, lit from the slope. Snow is computed per pixel from the season instead of being painted on the CPU. With the latitude $\ell_{uv} = 2|v - \frac{1}{2}|$, a relief term β and the land mask Λ,

$$\alpha_{\text{snow}} = \operatorname{sat}\Big(\frac{\ell_{uv} + k_\beta(\beta - 1) - \ell^*(t)}{w_s} + \tfrac{1}{2}\Big) \cdot \operatorname{sat}\big(k_\Lambda(\Lambda - \tfrac{1}{2})\big), \tag{23}$$

where the snow line $\ell^*(t)$ moves towards the equator in winter and towards the poles in summer.

6 Performance engineering

6.1 Methodology

Timings were taken with the C# code compiled for the .NET (Mono) runtime and run outside the engine after a warm-up pass. The browser build compiles the same code ahead of time to WebAssembly (IL2CPP), which is typically faster, so the figures are conservative.

6.2 Time-sliced generation

Executed in one call, generating a world on the larger grid took several hundred milliseconds in a single frame, a clearly visible freeze (Fig. 20a). The iterator of Section 3.11 is consumed under a fixed time budget per frame, so the same work spreads over a dozen or so frames while the loading animation continues (Fig. 20b). Map painting uses the same scheme.

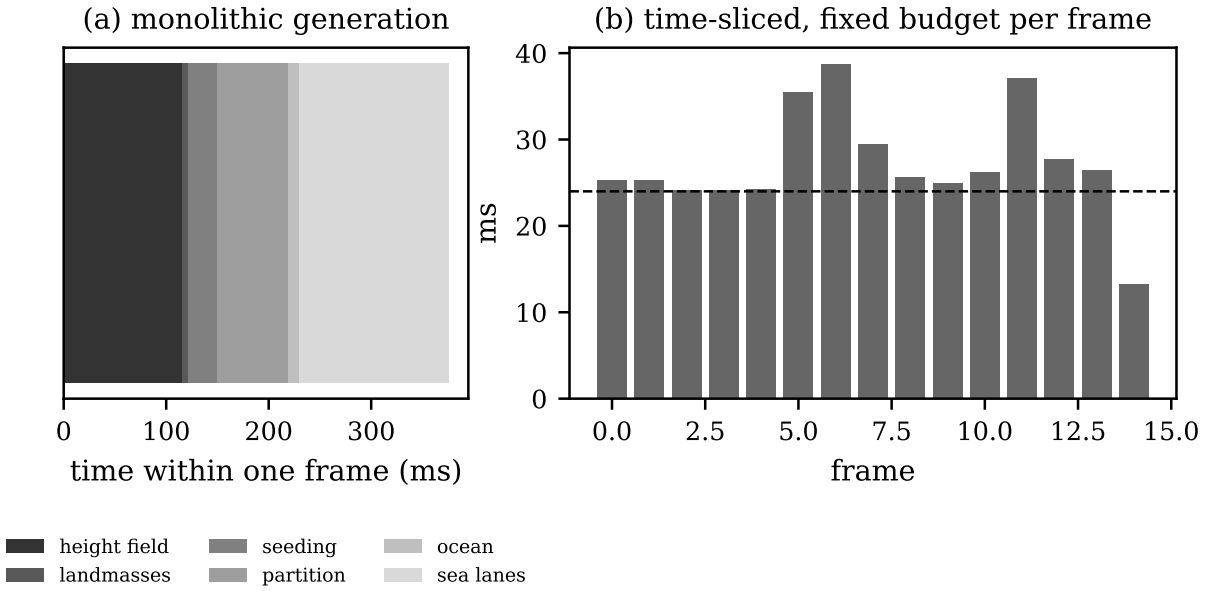


Figure 20: Generation on the larger grid. (a) Phase times when executed in a single frame. (b) Work per frame with time slicing.

6.3 Linear-time selection

Replacing the full sort of Section 3.4 by quickselect reduced the cost of finding the sea level by more than an order of magnitude on the larger grid while returning the identical value. It removed the longest uninterruptible step of the pipeline.

6.4 Tiled texture uploads

Uploading a whole data texture whenever a region is reached causes stalls at high simulation speed, when several regions may be reached within a second. Dynamic textures are therefore managed as grids of tiles. A change marks only the tiles covered by its bounding box (Fig. 21), and each frame at most a fixed number of dirty tiles is copied into a small staging texture and transferred into place with a GPU-side copy. This bounds the per-frame upload cost regardless of how much changed (Algorithm 4). Where copies between textures are unsupported, a full upload is performed at a limited rate.

Algorithm 4: Tiled upload of a dynamic texture, once per frame

Input: CPU copy of the texture, queue Q of dirty tiles, per-frame limit T

for $n \leftarrow 1$ **to** T ***while*** $Q \neq \emptyset$ **do**

  $t \leftarrow$ dequeue(Q); copy the pixels of t into the staging texture;

  upload the staging texture; GPU-copy it into the rectangle of t;



Figure 21: The tile grid of the cell texture. When a region is reached only its tiles (highlighted) are uploaded, a few per frame.

6.5 Further measures

Off-screen vehicles and carriers are culled before drawing. Background traffic grows with the square root of the number of regions rather than linearly. Interface rows, effects, vehicles and transient icons are pooled to avoid garbage collection, and interface texts are rebuilt only when their value changes. Measured on the larger grid, one simulated day of rules costs a few hundredths of a millisecond on average, a negligible share of the frame budget even at accelerated speed. Optimisation effort was therefore concentrated on rendering, texture traffic and generation, which dominate the frame time.

6.6 Delivery size

For browser delivery the size of the downloaded build matters as much as runtime speed, and it is dominated by assets rather than code. An automated editor step applies the following measures and produces a size report after each build, listing content by kind and the largest assets:

- **Textures** are stored with crunched block compression at the resolution at which they are actually displayed; mipmaps are kept only where an image is shown at many scales, and no CPU-side copies are retained.
- **Audio** is transcoded to the browser’s native compressed format at a reduced sample rate, in mono, and trimmed to the sections that are actually played (intro and loop).
- **Fonts** are baked into compact signed-distance-field atlases containing only the glyphs in use, and the source font files are excluded from the build.
- **Code** is reduced by managed-code and engine stripping and size-oriented ahead-of-time compilation, and the rendering pipeline is configured without features the game does not

use (high dynamic range, multisampling, shadows, depth and colour copies), with unused shader variants stripped.

- **Always-included content** (resource folders, splash images) is audited, so that nothing ships unless the build references it.

7 Validation

7.1 Determinism

Generation and simulation are pure functions of the seed and of the sequence of player decisions. Determinism is checked by hashing the generated grids and paths and comparing them across implementations (monolithic and time-sliced) and runs.

7.2 Balance with simulated players

A headless harness plays the real rules with scripted agents of increasing skill. The agents differ in how often they collect rewards, how often and how cleverly they buy upgrades (at random or following a plan), whether they act against laboratories and how often they use special actions. Each agent plays many generated worlds per difficulty level, and the harness reports win rates, session lengths and the state of the countermeasure at the end. The results confirm the intended properties: inactive play never wins, any active play usually wins at the standard difficulty, success at the higher difficulty depends clearly on activity and planning, and session lengths stay within the intended range.

7.3 Runtime

Generation, simulation and frame times are measured with the same harness and with the engine profiler, as described in Section 6. Time slicing keeps every frame close to the budget during generation, and the cost of the rules is negligible compared with rendering.

8 Discussion and limitations

The model favours legibility over epidemiological realism. Within a region the dynamics are deterministic and logistic, without recovery or spatial structure; spatial heterogeneity inside a region exists only in the visual model of Section 4.4, which is decoupled from the rules by construction. Transmission is evaluated per link and per day, so the waiting time for a jump is geometric, and correlations between simultaneous jumps are not modelled. The rubber band of Eq. (21) deliberately trades strict monotonicity of difficulty for close contests; the simulated players show that it does not invert the ordering of skill. Runtime figures were measured outside the browser runtime and are therefore indicative.

9 Conclusion

A single seed drives a deterministic pipeline of classical algorithms: value noise and fBm, order statistics, breadth-first labelling, farthest-point sampling, a noisy multi-source Dijkstra partition, union–find and weighted A*. It produces varied, connected and playable worlds that are delivered without frame stalls by time slicing. On top of it, a compact metapopulation model with logistic growth, stochastic link transmission, endogenous detection and an adaptive

countermeasure yields sessions whose difficulty can be tuned and verified automatically with simulated players. A threshold-field visual model turns one number per region into an organic, creeping colony that the GPU draws at negligible cost. Together these choices meet the goals of replayability, legible rules and fluid browser performance.

References

[1] K. Perlin. An image synthesizer. *ACM SIGGRAPH Computer Graphics*, 19(3):287–296, 1985.

[2] A. Lagae, S. Lefebvre, R. Cook, T. DeRose, G. Drettakis, D. S. Ebert, J. P. Lewis, K. Perlin and M. Zwicker. A survey of procedural noise functions. *Computer Graphics Forum*, 29(8):2579–2600, 2010.

[3] B. B. Mandelbrot. *The Fractal Geometry of Nature*. W. H. Freeman, 1982.

[4] D. S. Ebert, F. K. Musgrave, D. Peachey, K. Perlin and S. Worley. *Texturing and Modeling: A Procedural Approach*, 3rd ed. Morgan Kaufmann, 2003.

[5] C. A. R. Hoare. Algorithm 65: Find. *Communications of the ACM*, 4(7):321–322, 1961.

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.

[7] T. F. Gonzalez. Clustering to minimize the maximum intercluster distance. *Theoretical Computer Science*, 38:293–306, 1985.

[8] Y. Eldar, M. Lindenbaum, M. Porat and Y. Y. Zeevi. The farthest point strategy for progressive image sampling. *IEEE Transactions on Image Processing*, 6(9):1305–1315, 1997.

[9] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

[10] G. Voronoi. Nouvelles applications des paramètres continus à la théorie des formes quadratiques. *Journal für die reine und angewandte Mathematik*, 133:97–178, 1908.

[11] R. E. Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.

[12] P. E. Hart, N. J. Nilsson and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[13] G. M. Chaikin. An algorithm for high-speed curve generation. *Computer Graphics and Image Processing*, 3(4):346–349, 1974.

[14] W. O. Kermack and A. G. McKendrick. A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society of London A*, 115:700–721, 1927.

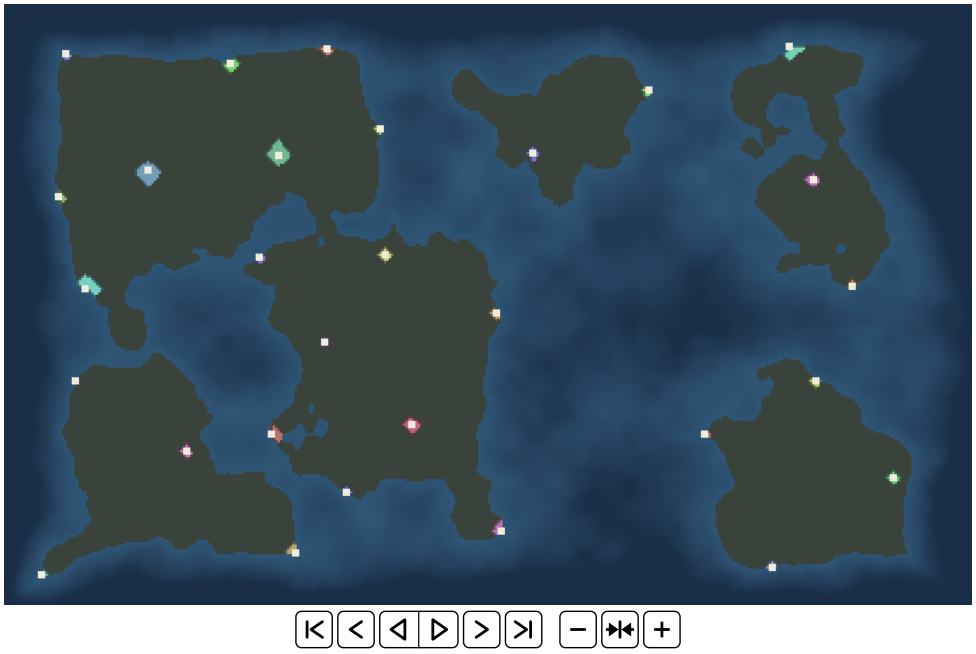
[15] P.-F. Verhulst. Notice sur la loi que la population suit dans son accroissement. *Correspondance mathématique et physique*, 10:113–121, 1838.

A Symbols

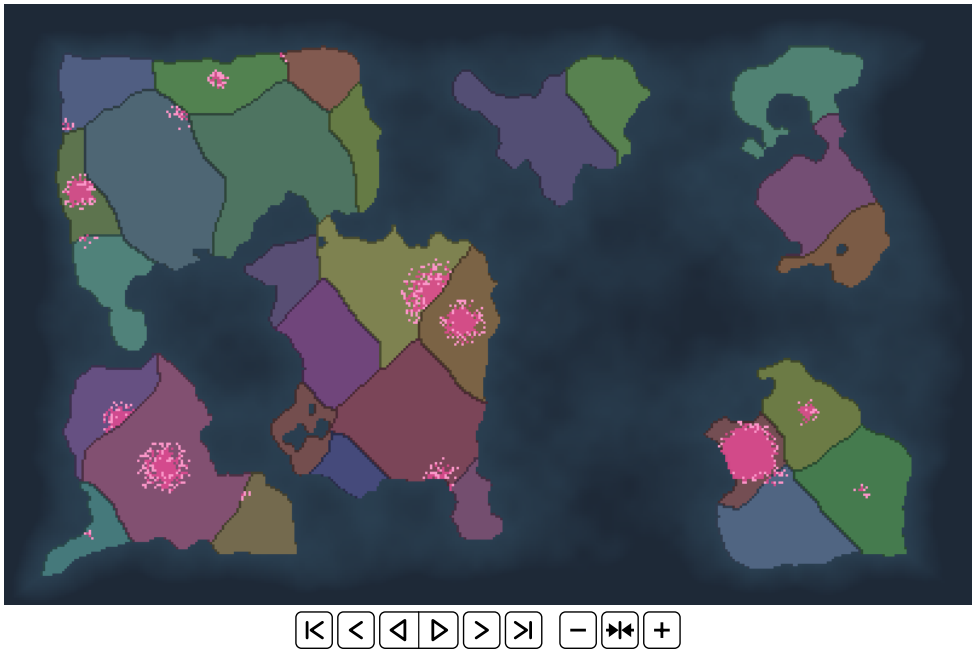
Symbol	Meaning
W, H, N	grid width, height, number of cells
$N(x, y)$, $F(x, y)$	value noise, fractional Brownian motion
h, θ, λ	height, sea level, target land share
N_R	number of regions
$w(q)$	partition step cost
ς, ω, $\hat{h}$	distance to shore, coastal penalty, A* heuristic
I_i, P_i, f_i	infected, population, coverage of region i
W (in Section 4)	world coverage
ρ	global pace
b_i, S_i	growth rate, seasonal factor
p_{ab}, τ, ℓ	transmission probability, base rate, leak factor
ϑ_c, v_i, η	cell threshold, region visibility, grain weight
A, C, E	suspicion, countermeasure, rubber-band factor
$\mathcal{H}$	wealth-weighted healthy share
π_i	paranoia of region i
ν	colony noise

B Supplementary animations

The animations below are embedded in the PDF and play in viewers that support JavaScript animation, such as Adobe Acrobat Reader (click to play); other viewers show the first frame. The same animations are provided as separate GIF files.



Animation S1. Multi-source Dijkstra growth of the regions with the noisy cost of Eq. (9).



Animation S2. A simulated session from early spread to saturation, drawn with the threshold model.



Animation S3. One region filling in as its visibility v rises past the per-cell thresholds ϑ_c of Eq. (16).